

NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Data Science and Business Analytics"

UDC 336.76:004.9

Research Project Report on the Topic:
Financial Market Stability under Heterogeneous Information Speeds: An
Agent-Based Modeling Approach

Submitted by the Students:

Group БПАД246, 2nd year of study

Ravilova Arina Kharisovna

Approved by the Project Supervisor:

Lukyanchenko Petr Pavlovich

Head of Lab

Faculty of Computer Science, HSE University

Moscow 2026

Abstract

This project examines how differences in the speed of information acquisition by market participants affect the stability of financial markets. The study focuses on three interrelated hypotheses: that heterogeneity in traders' latency creates a tipping point between normal liquid market conditions and a "toxic" liquidity crisis; that updating information in event time as opposed to real calendar time can alter market resilience and shift the boundary at which crises occur; and that combining information delays with liquidity constraints produces volatility clustering, which becomes more severe during crisis regimes. To investigate these hypotheses, we propose to develop an agent-based model of a financial market with fast and slow agents, where we can vary information delays and the timing of updates. The relevance of this research is underscored by phenomena like the Flash Crash of 2010 and other extreme volatility events associated with high-frequency trading. By comparing insights from existing literature and conducting simulation experiments, we anticipate identifying critical thresholds of latency differences and liquidity withdrawal that precipitate instability. The expected results will contribute to understanding how modern markets can suddenly transition from stable to chaotic, and they will inform potential innovations in market design to improve stability.

Данный проект исследует, как различия в скорости получения информации участниками рынка влияют на стабильность финансовых рынков. Исследование сосредоточено на трех взаимосвязанных гипотезах: что гетерогенность задержек у трейдеров создает критическую точку, разделяющую нормальные условия ликвидности и «токсичный» кризис ликвидности; что обновление информации в событийном времени по сравнению с реальным временем может изменить устойчивость рынка и сдвинуть порог наступления кризиса; и что сочетание задержек в распространении информации с ограниченной ликвидностью приводит к кластеризации волатильности, которая усиливается в режимах кризиса. Для проверки этих гипотез предлагается разработать многоагентную модель финансового рынка с быстрыми и медленными агентами, в которой можно варьировать задержки информации и режим времени обновления. Актуальность исследования подтверждается такими явлениями, как «молниеносный крах» 2010 года и другие эпизоды экстремальной волатильности, связанные с высокочастотной торговлей. Опираясь на результаты существующей литературы и проводя имитационные эксперименты, мы ожидаем выявить критические уровни различий в скорости и отзыва ликвидности, которые приводят к нестабильности. Ожидается, что результаты работы позволят лучше понять, как современные рынки могут внезапно переходить от ста-

бильности к хаосу, а также помогут в разработке новых мер по улучшению стабильности рынков.

Keywords: Financial market stability; High-frequency trading (HFT); Agent-based modeling; Latency heterogeneity; Liquidity risk; Volatility clustering; Information delay; Market microstructure; Flash Crash.

Contents

Abstract	2
1 Introduction	5
2 Literature overview	7
3 Report	13
3.1 The Structure of the Simulator and the Market Model	13
3.1.1 Exchange Agent and the Order Book	13
3.1.2 Dividend Process	14
3.1.3 Trading Agents	14
3.1.4 Simulation Loop	16
3.1.5 Exogenous Events	17
3.1.6 Data Recording and Derived Indicators	17
3.1.7 Summary	17
3.2 Experimental Design and Model Modifications	18
3.2.1 H1: Latency Heterogeneity and the Tipping Point	18
3.2.2 H2: Calendar Time versus Event Time	20
3.2.3 H3: Information Delay, Liquidity Constraints, and Volatility Clustering . .	22
3.2.4 Summary of Design Principles	23
3.3 Hypothesis H1: Experimental Results	24
3.3.1 Hypothesis Statement and Metrics	24
3.3.2 Iterative Experimental Design	25
3.3.3 Results	30
3.3.4 Conclusions for H1	32
References	34

1 Introduction

Financial markets can abruptly shift from stable and liquid to turbulent and illiquid, as dramatically illustrated by events like the 2010 "Flash Crash." A growing body of research suggests that differences in how quickly traders acquire and act on information may be a key factor driving such instability. In modern electronic markets, high-frequency trading firms leverage ultralow latency to anticipate and outrun slower investors. This latency heterogeneity can lead to situations where liquidity suddenly vanishes and prices crash before most participants can respond. The subject area of this project is the stability of financial markets under asynchronous information flows – in particular, we focus on how heterogeneous information speeds and delays affect market liquidity and volatility. The problem can be described informally as understanding why markets sometimes “flip” from orderly trading to a frantic collapse when some traders are faster than others, and how changing the timing mechanism of information updates or the availability of liquidity might prevent such crises.

Formally, we consider an agent-based modeling approach to capture the interactions between fast and slow traders in a limit order market. The research problem is to determine how the speed at which different agents receive market information and react influences overall market stability. Key aspects include: identifying tipping points where latency differences lead to a breakdown in liquidity; examining the effect of using event-time updates versus standard clock time on market resilience; and investigating how information delays combined with liquidity constraints can generate volatility clustering and extreme price fluctuations characteristic of crises.

We will implement our agent-based model using a one-dimensional ABM simulation framework available as open-source code. This simulator will be extended to incorporate heterogeneous information-acquisition and to simulate crisis regimes such as extreme liquidity shock scenarios akin to a flash crash. These modifications will enable us to systematically examine how latency differences and extreme market stress jointly affect overall market stability.

The relevance and significance of this topic are evident given the increasing automation and speed of trading. Markets today operate on timescales beyond human reaction, raising concerns that the market’s stability might be fragile under algorithmic “arms races.” Understanding these dynamics is important not only academically but also for regulators and practitioners aiming to ensure orderly markets. Innovative aspects of this project include bridging insights from market microstructure theory, high-frequency trading empirical analyses, and complex system modeling. We will synthesize these perspectives into an agent-based simulation that, unlike traditional equilibrium models, can capture non-linear crisis dynamics. By doing so, the work innovates in testing

how alterations in market design might avert instability.

The anticipated results of this research include: identification of quantitative thresholds beyond which a liquid market becomes unstable; a demonstration of how using an event-driven clock for information can increase market robustness by removing predictable arbitrage opportunities; and insights into how delayed information propagation and limited liquidity can reproduce stylized facts like clustered volatility and fat-tailed price changes. These outcomes will be initially described in this plan as hypotheses and will later be validated through simulation experiments. Although the actual results are yet to be obtained, we will indicate in the Abstract and Introduction what we expect to find, with a disclaimer that these are so far planned outcomes. Overall, this project will position itself among contemporary studies of market stability by providing a modeling framework to explore policies for safer market operation in the age of HFT. The next section provides an overview of related research, positioning our work in the context of existing findings and supporting the three central hypotheses guiding our study.

2 Literature overview

Research on market stability, high-frequency trading, and agent-based modeling provides a foundation for each of our three hypotheses. Rather than a list of summaries, this review analytically groups the 16 sources by the key insights they contribute to our hypotheses: that latency heterogeneity can trigger a tipping point between stable liquidity and a toxic-flow crisis; that event-time vs. calendar-time information updates alter market resilience and can shift the crisis boundary; and that information delays coupled with liquidity constraints lead to volatility clustering, which intensifies during crises. We discuss how each group of sources supports these ideas and how, in combination, they inform the design of our agent-based model. Throughout the review, connections to agent-based modeling, high-frequency trading, and liquidity dynamics are highlighted, demonstrating how our future work builds on and goes beyond these studies.

Several studies indicate that when some traders are significantly faster than others, markets can reach a critical point where liquidity provision breaks down. Latency heterogeneity means informed traders can react and trade before slower counterparts, potentially generating toxic order flow that market makers struggle to absorb. Easley et al. introduce the concept of flow toxicity – the probability that incoming order flow is from informed traders – and show that excessive toxicity can cause a liquidity crisis[2]. In their analysis of the Flash Crash, they found that if the fraction of trades by informed HFTs rises above expectations, market makers incur losses and quickly hit risk limits. At that point they withdraw from the market, turning from liquidity suppliers into liquidity demanders[1]. This withdrawal creates a self-reinforcing spiral, as liquidity vanishes on one side of the order book, prices move even more violently, causing further losses for market makers and ultimately a “liquidity crash.” In other words, beyond a certain latency advantage held by informed traders, liquidity provision becomes unviable and the market “tips” into instability.

Empirical and simulation studies corroborate this tipping-point behavior. Regulatory investigations of the May 6, 2010 Flash Crash by Kirilenko et al. observed that HFT firms initially provided liquidity but then sold aggressively and pulled out when volatility spiked, exacerbating the crash[5]. In fact, the abrupt collapse happened within seconds, absent any fundamental news, highlighting how purely endogenous latency dynamics can trigger a crisis. Johnson et al. documented the emergence of a new ultrafast “machine ecology” in financial markets around 2006-2011, finding an explosion of sub-second extreme price events attributable to algorithmic trading[4]. They report an abrupt transition to an all-machine regime where human intervention is too slow, and note that the proliferation of these mini flash crashes correlates with the onset of

the 2008 financial crisis. This suggests that as algorithmic trading dominance grew, the market became more prone to sudden liquidity evaporation. The notion of predatory trading further illuminates this: when a large trader is forced to sell, “predators” also sell and withdraw liquidity, leading to price overshooting and systemic risk[9]. In other words, the market is most illiquid precisely when liquidity is most needed – a hallmark of crisis. Carlin et al. similarly contrast cooperative vs. predatory trading in liquidity crises, concluding that in non-cooperative scenarios traders rush to exploit others’ distress, accelerating market crashes[10]. These works underscore that heterogeneous speeds and strategic order-flow anticipation create a fragility: once informed trading crosses a threshold, market stability can flip, as liquidity providers either cannot keep up or choose to flee to avoid losses.

Notably, agent-based models have managed to reproduce such phenomena by explicitly modeling heterogeneous agent speeds and decision cycles. Bookstaber and Paddrik developed an ABM with different classes of traders, slow liquidity suppliers vs. faster liquidity demanders, and showed how crisis-induced illiquidity can arise from their interactions[6]. Their simulation of a liquidity shock demonstrated that if market makers’ capacity to intermediate is exceeded, liquidity can dry up entirely. Crucially, their model revealed that the presence of agents operating on different timescales was a key factor in crisis dynamics. When the fast agents dumped assets, slower agents could not step in timely, and the market experienced a dramatic liquidity shortfall. Consistent with empirical observations, this ABM produced scenarios where market makers reached their risk limits and stopped providing quotes, mirroring the “tipping point” of real markets. The model also captured a continuum from stable conditions to unstable, toxic-flow conditions beyond a critical point – effectively validating Hypothesis 1. These findings position our work, we will extend such agent-based frameworks by incorporating explicit information delays and different update mechanisms, to further explore how and when latency heterogeneity causes instability. Our hypothesis is that there exists a critical degree of latency asymmetry beyond which even a well-capitalized market maker agent cannot prevent a collapse. The literature reviewed here strongly supports that hypothesis, providing both the conceptual mechanism and evidence from real markets and simulations that such a tipping point is real[6]. Our model will build on these insights, using flow toxicity measures and market maker inventory dynamics to detect when the system crosses into an unstable regime.

While latency differences create instability, another line of research examines how the timing method of information updates, event-driven vs. clock-driven, can influence market outcomes. Traditional markets operate in continuous calendar time, where orders arrive at any moment and information is disseminated continuously. In such a setting, faster traders exploit even the

smallest temporal edge. An alternative is to use discrete event time, which might level the playing field. Our second hypothesis posits that markets updating in event time may be more resilient, effectively shifting the threshold of crisis by reducing the advantage of speed.

Easley et al. provide a clear illustration of the difference between chronological time and event time through their concept of the “volume clock”. They argue that measuring market activity in equal volume intervals can better detect stress buildup. For example, in their analysis, price change distributions look very different when sampled every fixed number of trades versus every fixed number of seconds[3]. Importantly, they observed that many trading algorithms of slower traders operate on the time-of-day clock, which creates predictable surges that fast traders exploit. Easley et al. note that high-frequency traders effectively “live” in volume time – they react as soon as a pattern of trading is detected, rather than waiting for the next clock interval. Because most institutional algorithms were designed in calendar time, their behavior becomes a footprint that HFTs anticipate and capitalize on. This dynamic not only gives an edge to the fastest players but can also contribute to instabilities if a large number of orders flood the market simultaneously at predictable times. The literature suggests that resilience might be improved by switching to event-time processing, where updates happen only after a certain volume of trades or at discrete frequent intervals, breaking the synchronicity that HFTs prey upon.

Supporting this, Budish et al. famously argued that the continuous limit order book is a flawed design in the presence of latency arbitrage, and proposed frequent batch auctions as a remedy[11]. They theoretically showed that batching trades in discrete intervals would eliminate the arbitrage rents from speed and diminish the arms race, without hurting allocative efficiency. Empirical and agent-based studies back up these claims. Wah and Wellman built a strategic agent-based model of a fragmented market with latency arbitrage opportunities, where a single security trades on two exchanges with slightly out-of-sync information. In their simulations, an infinitely fast arbitrageur could exploit price differences between venues caused by communication delays, yielding profit at the expense of market efficiency. Their key result was that the presence of such a latency arbitrageur “reduces total surplus and negatively impacts liquidity” in the market[7] – essentially, overall welfare goes down when one agent can consistently win the race between exchanges. This validates the idea that asynchronous information harms liquidity and fairness. Crucially, Wah and Wellman demonstrate that switching to periodic call auctions across the exchanges not only removes the arbitrage opportunity but actually improves liquidity and efficiency by aggregating orders[7]. In equilibrium experiments, the batch auction design achieved higher surplus because it neutralized the speed advantage and encouraged more stable trading strategies.

These findings lend strong support to Hypothesis 2: the mode of information updating can

indeed shift market stability. A continuous-time market, where faster agents constantly race slower ones, tends to suffer from sniping and “toxic” volatility – small delays can lead to big price discrepancies that invite predatory trading. By contrast, an event-time or discretized market introduces a minimum temporal granularity that all but eliminates certain micro-arbitrage opportunities. In practical terms, this can raise the threshold at which the market hits a crisis. For example, if orders are aggregated even every 0.1 second, the fastest traders cannot exploit differences at the microsecond level, possibly preventing a cascade of rapid-fire order cancellations that amplify volatility. Our literature review thus indicates that markets are more robust when operating on an event-time basis, aligning with theoretical proposals and simulation evidence. In our agent-based model, we plan to incorporate this by running experiments under two regimes, a continuous-time LOB vs. a discrete-time LOB. We expect to observe that the continuous regime is more prone to dramatic liquidity drops given the same shock or the same degree of agent heterogeneity, whereas the event-time regime buffers those effects. The sources reviewed show why. Synchronized batch processing forces interaction at set intervals, preventing fast traders from iteratively eroding liquidity between slower traders’ actions. Additionally, measuring risk indicators in event time could give earlier warning of stress than traditional time-based metrics. Overall, the consensus is that adapting the market’s time structure can shift the “crisis boundary” – potentially delaying or mitigating a toxic-flow regime. We will test this hypothesis by seeing if our simulated market’s critical tipping point moves when switching from a continuous to a discrete-time mechanism.

Our third hypothesis ties together information delays and liquidity constraints, positing that their combination produces volatility clustering, periods of high volatility followed by more high volatility, which becomes more pronounced during crises. In financial markets, volatility clustering is a well-known stylized fact, often attributed to waves of information or herd behavior. We suspect that when some agents experience delays in receiving information and liquidity is not infinitely deep, the result is intermittent bursts of trading activity and price jumps as delayed reactions catch up, and as liquidity providers adjust. During normal times, these delays might cause mild oscillations, but during stressed conditions, delayed information combined with low liquidity can lead to intense, self-perpetuating volatility spikes.

The literature provides both empirical evidence and ABM support for this mechanism. Zhou et al. (2022) directly study the roles of information delay and liquidity in an agent-based model of a stock market[8]. They introduce a “delay agent-based model” where agents respond to news with a certain lag, and they analyze market stability using escape-time metrics from physics. Their findings are somewhat surprising yet insightful:

1. There exists an optimal amount of information delay that maximizes stability – a small

delay can smooth out overreactions and leads to the longest mean time before the market escapes a stable equilibrium. Intuitively, if everyone reacts instantly, feedback loops may form immediately; a bit of delay can dampen these loops as not all agents dog-pile on new information at once.

2. Increasing overall liquidity beyond a point actually weakens stability. This is counterintuitive, as one might assume more liquidity is always stabilizing; however, the authors explain that abundant liquidity can encourage more aggressive trading and risk-taking, which under certain conditions makes crashes more severe when they do occur.

3. Both delay and liquidity show non-monotonic effects on stability, meaning too little or too much of either can reduce stability. In other words, the relationship is an inverted U-shape: a moderate information delay is stabilizing, but very long delays would be destabilizing; similarly, moderate liquidity is good, but extremely high liquidity might create an illusion of safety that leads to abrupt corrections. Crucially, Zhou et al. observe that information delay tends to enhance stability in their simulations, suggesting that markets where not everyone chases information at the same millisecond may avoid some volatility. However, they also note the presence of a “worst market noise” level that maximally destabilizes the system—corresponding to an optimal mix of delay and liquidity that produces the fastest escape from stability.

Volatility clustering itself is reproduced in many agent-based models as a result of feedback mechanisms. Kirman and Teyssière (2002) showed that herding behavior in an ABM could generate long-memory in volatility[14]. Similarly, the ABM by Bookstaber & Paddrik discussed earlier demonstrates volatility clustering arising from market-maker and liquidity-supplier interactions[6]. In their model, when volatility increases and market makers become cautious, they pull back liquidity, which causes subsequent price changes to be larger and more correlated, yielding a slowly decaying autocorrelation of absolute returns. They identify that the decision of liquidity providers to supply more or less liquidity is a herding-like variable: when many providers withdraw at once, volatility remains high until they come back. Figure 7 of their paper confirms that the model’s simulated returns exhibit the same volatility clustering as real market data. This aligns with our hypothesis that liquidity constraints combined with delayed or cautious re-entry into the market produce volatility clustering. Essentially, after an initial large move, liquidity is slow to return, so prices continue to swing widely – a clustered volatility regime. Only after some time does stability return, breaking the cluster.

The interplay of information delays and liquidity is also evident in crisis scenarios. During major market events, information often arrives unevenly, and liquidity can dry up. This was seen in 2008, news about mortgage losses spread in stages while banks’ balance-sheet constraints severely limited liquidity, resulting in protracted high volatility. Our hypothesis posits that in a

simulated environment, if we introduce a lag for some agents in receiving price signals or news, and simultaneously impose constraints on how much inventory market-makers can absorb, the model will produce clustered volatility and heavy tails in the return distribution – especially once a critical stress threshold is passed. The literature we reviewed provides a roadmap, Zhou et al. indicate what ranges of delay and liquidity might be critical, and Bookstaber’s ABM confirms that heterogeneous reaction times and finite liquidity indeed create the conditions for clustered volatility[6].

In summary, Hypothesis 3 is well-supported by existing sources, information delays can both stabilize and destabilize, depending on context, but when combined with liquidity shortages, they tend to cause persistent high volatility as the market struggles to find equilibrium. The clustering intensifies in crisis regimes because delays in reaction or recapitalization mean the market cannot quickly correct mispricings, leading to successive large swings. Our model will extend these insights by allowing us to toggle information delays for different agent groups and simulate liquidity provider behavior under capital constraints. We will measure volatility clustering in our outputs and expect to see stronger clustering in scenarios where agents face significant delays and where market-maker inventory is limited, versus a baseline with no delays and ample liquidity. This exercise will also connect back to the toxic flow idea, volatility clustering can be seen as a symptom of an ongoing toxic scenario where order flow is one-sided or imbalanced over an extended period. Indeed, Easley et al. found that their VPIN toxicity metric could anticipate volatility events better than traditional volatility indices, implying that persistent order flow imbalances lead to clustered volatility[1].

By positioning our work in the context of these three hypothesis-driven literatures, we ensure that our planned model addresses the most salient mechanisms identified by prior research: the point of liquidity collapse from speed disparities, the mitigation of instability through alternative timing, and the generation of realistic volatility patterns through delays and feedback. In the next section, we outline how we intend to build and use our simulation model to verify these insights and explore new ground.

3 Report

3.1 The Structure of the Simulator and the Market Model

We use a discrete multi-agent simulation system with a centralized limit order book and endogenous price dynamics. The architecture of the model is built around three main components: the stock exchange (**ExchangeAgent**), a set of trading agents (**Trader** and its derived classes), and the simulation engine (**Simulator**). Each component is described below.

3.1.1 Exchange Agent and the Order Book

The exchange implements the order book as two sorted doubly linked lists (**OrderList**), representing the bid and ask sides respectively. Each node in the list is an **Order** object containing a price, quantity, order type, and a back-reference to the trader who placed the order. Orders within each list are maintained in best-offer-first order: for bids the highest price appears first, for asks the lowest. The comparison operators of **Order** are overloaded so that “less than” always means “better offer,” regardless of side. Insertion preserves this invariant in $O(n)$ time, while appending and removal of a known node are $O(1)$.

The order book is initialized with V randomly generated orders (default $V = 1000$) whose prices are drawn from two normal distributions centered at $p_0 - \sigma$ and $p_0 + \sigma$ (default $p_0 = 100$, $\sigma = 25$), with random integer quantities drawn uniformly from $[1, 5]$. Orders with price above p_0 are placed on the ask side; orders at or below p_0 are placed on the bid side. The reference market price at each iteration is defined as the mid-price:

$$p_t = \frac{p_t^{\text{bid}} + p_t^{\text{ask}}}{2},$$

where p_t^{bid} and p_t^{ask} are the best bid and best ask quotes.

Order execution is handled by the **OrderList.fulfill** method, which walks the opposing side of the book from best to worst offer, matching quantity until the incoming order is fully filled or no more compatible orders remain. All cash and asset transfers — including the application of transaction costs — are executed directly within this method upon each partial fill.

The exchange supports three order types exposed to traders: limit orders (placed into the book and potentially matched against the opposite side if the price crosses the spread), market orders (immediately matched against the opposite side starting from the best available price), and cancellation orders (which remove a previously placed order from the book).

3.1.2 Dividend Process

The exchange maintains a stochastic dividend process implemented as a list of 100 pre-generated future dividend values (the *dividend book*). At each simulation step, the oldest dividend is consumed as the current payment, a new value is appended, and the list shifts forward. New dividends are generated multiplicatively:

$$d_{t+1} = \max(d_t \cdot e^{\varepsilon_t}, 0), \quad \varepsilon_t \sim \mathcal{N}(0, \sigma_d^2),$$

where $\sigma_d = 5 \times 10^{-3}$ by default. The $\max(\cdot, 0)$ constraint ensures non-negative dividends. The initial dividend value equals $r_f \cdot p_0$, where r_f is the risk-free rate (default 5×10^{-4} per iteration). Fundamentalist agents can observe a finite prefix of the dividend book controlled by their `access` parameter.

3.1.3 Trading Agents

All agents share a common base class `Trader`, which stores cash, assets (number of shares held), and a list of active orders. The base class exposes five primitive actions: `_buy_limit`, `_sell_limit`, `_buy_market`, `_sell_market`, and `_cancel_order`. There are four strategic agent types and one auxiliary agent type:

Random agents generate noisy orders to approximate background liquidity. On each call, the agent draws a random state and acts as follows: with probability 15% it places a market order (buy or sell with equal probability); with probability 35% it places a limit order at a price drawn either uniformly within the spread (with 35% chance) or outside the spread by an exponentially distributed offset (with 65% chance); with the remaining probability it cancels a randomly selected previously placed order. Order quantities are drawn uniformly from $[1, 5]$.

Fundamentalist agents estimate the present value of the asset using a constant-dividend discount model. Given a vector of n observable future dividends $(d_1, \dots, d_n)$ and risk-free rate r_f , the estimated fundamental price is:

$$p_f = \sum_{i=1}^{n-1} \frac{d_i}{(1 + r_f)^i} + \frac{d_n/r_f}{(1 + r_f)^{n-1}},$$

where the last term is the perpetuity value of dividends beyond the forecast horizon. The number of observable dividends is controlled by the `access` parameter (default 1). When the market price

deviates from p_f , the agent submits buy or sell orders; the quantity is proportional to the relative mispricing $|p_f - p|/p$, capped at 5 units. Approximately 45% of calls result in order cancellation rather than submission.

Chartist agents hold a binary sentiment state (Optimistic or Pessimistic) and trade in the direction of that sentiment: optimists buy, pessimists sell, using a mix of market orders (15%), limit orders (35%), and cancellations (remaining). The sentiment evolves at each iteration via a probabilistic switching rule. Defining $x = (n_O - n_P)/n_C$ as the normalized sentiment imbalance among all chartists, Δp as the most recent price change, and p as the current price, the opinion index is:

$$U = a_1 \cdot x + \frac{a_2}{v_1} \cdot \frac{\Delta p}{p}.$$

An Optimistic agent switches to Pessimistic with probability $v_1 \cdot (n_C/N) \cdot \exp(U)$, and vice versa with probability $v_1 \cdot (n_C/N) \cdot \exp(-U)$, where N is the total number of traders. We note that this sign convention means that when prices are rising ($U > 0$), Optimists have a *higher* probability of switching to Pessimistic, producing *contrarian* rather than trend-following behavior. This design choice in the baseline code is discussed and addressed in Section 3.3.

Universalist agents inherit from both **Fundamentalist** and **Chartist** via multiple inheritance and can switch between these two strategies at each iteration. On each call, the agent executes the strategy corresponding to its current type. Strategy switching is governed by a utility-based rule that compares expected fundamental returns against realized average returns across all agents:

$$U_1 = a_3 \left(\frac{r + \frac{1}{v_2} \Delta p}{p} - \bar{R} - s \left| \frac{p_f - p}{p} \right| \right), \quad U_2 = a_3 \left(\bar{R} - \frac{r + \frac{1}{v_2} \Delta p}{p} - s \left| \frac{p_f - p}{p} \right| \right),$$

where $r = p_f \cdot r_f$ is the expected dividend return, $\bar{R}$ is the cross-agent mean equity return, and s controls the sensitivity to mispricing opportunities. Both U_1 and U_2 are clamped to $[-100, 100]$ to prevent numerical overflow in the exponential. Transition probabilities depend on the current type and sentiment, with the switching frequency scaled by $v_2 \cdot n_{\text{type}}/N$.

MarketMaker maintains two-sided quotes and manages inventory within a soft band $[-\text{softlimit}, +\text{softlimit}]$. At each call, all previously placed orders are cancelled. The bid and ask volumes are set as

$$q_{\text{bid}} = \max(0, \text{ul} - 1 - a_t), \quad q_{\text{ask}} = \max(0, a_t - 11 - 1),$$

where a_t is the current inventory and `ul`, `ll` are the upper and lower bounds. Prices are offset from the current spread by an inventory-dependent skew

$$\delta = -(p^{\text{ask}} - p^{\text{bid}}) \cdot a_t / \text{softlimit},$$

which widens the quote on the side where inventory is concentrated. When either volume reaches zero, the agent sets an internal `panic` flag. We note, however, that in the current code the rebalancing logic in the panic branch checks for `None` rather than zero, so the one-sided market-order rebalancing is never triggered; the panic flag is set but not operationally acted upon.

3.1.4 Simulation Loop

Simulation is implemented as a fixed-length iterative process of N steps. At each iteration t the following actions are performed in sequence:

- 1 **Exogenous events.** All registered events are checked; those scheduled at t modify the model state (see below).
- 2 **State capture.** `SimulatorInfo.capture()` records the current market and agent-level variables.
- 3 **Behavioral update.** The strategy and sentiment of `Universalist` and `Chartist` agents are updated based on the newly captured state.
- 4 **Trading.** The list of traders is uniformly shuffled, and each agent's `call()` method is invoked, which may place, cancel, or execute orders.
- 5 **Payments.** Each trader receives dividend income proportional to assets held (`cash +=  $a_t \cdot d_t$`) and risk-free interest on cash (`cash +=  $\text{cash} \cdot r_f$`). Negative dividends and negative cash balances are permitted by the model.
- 6 **Dividend generation.** The exchange shifts the dividend book forward by one step, generating the next future dividend.

The trading dynamics of the model are thus endogenous, while the dividend process and intervention events are exogenous.

3.1.5 Exogenous Events

The system supports seven types of exogenous events, each activated at a pre-specified iteration:

- **MarketPriceShock** — shifts all order prices in the book by a fixed amount Δp ;
- **FundamentalPriceShock** — adjusts the entire dividend book by $\Delta p \cdot r_f$, thereby shifting the fundamental value;
- **LiquidityShock** — creates a pseudo-trader and executes a one-sided market order of specified volume, directly removing depth from the book;
- **InformationShock** — changes the **access** parameter of all Fundamentalist and Universalist agents;
- **MarketMakerIn / MarketMakerOut** — dynamically adds or removes MarketMaker agents during the simulation;
- **TransactionCost** — changes the exchange’s transaction cost parameter.

3.1.6 Data Recording and Derived Indicators

The **SimulatorInfo** object records per-iteration time series of: market prices, bid-ask spreads, dividends, order book depth (number of bid and ask orders), and for each agent — equity, cash, asset holdings, current strategy type, and sentiment (for Chartist agents). Per-agent returns are computed as the relative change in equity between consecutive iterations.

From these raw series, the following derived indicators are available: rolling stock returns and abnormal returns (return minus r_f), return volatility over a window, rolling price volatility, the relative bid-ask spread as a liquidity proxy $((ask - bid)/p)$, and the fundamental value trajectory given a specified **access** level.

The **states** module provides statistical procedures for identifying market regimes in rolling windows. Each window of a specified size is classified as one of five states — *stable*, *trend*, *panic*, *disaster*, or *mean-reverting* — based on OLS trend tests applied to the rolling price and volatility series and threshold comparisons on volatility levels.

3.1.7 Summary

The model is a software-implemented multi-agent system with a centralized limit order book, a log-normal stochastic dividend process, five types of strategically heterogeneous agents,

and an event intervention module. The simulation operates in discrete time and generates both microstructure indicators (spreads, order book depth, liquidity) and macrostructure indicators (volatility, returns, market states) for subsequent analysis.

3.2 Experimental Design and Model Modifications

We verify the three hypotheses by introducing modifications to the existing architecture (`ExchangeAgent` + `Trader` classes + `Simulator` + Event module + `SimulatorInfo`) that satisfy three design principles: (i) they can be implemented without changing the base logic of the limit order book; (ii) they have a clear correspondence to the economic content of each hypothesis; and (iii) they lead to formally definable and quantitatively comparable metrics. For each hypothesis we specify: which parts of the code are modified, which parameters vary across the scenario grid, which indicators are used for verification, and which criteria determine confirmation or rejection.

3.2.1 H1: Latency Heterogeneity and the Tipping Point

To test the hypothesis that a critical proportion of fast traders triggers a transition from normal to “toxic” liquidity, we must introduce a systematic time advantage for one group of agents. In the baseline implementation, all traders are called once per iteration in random order after `random.shuffle(self.traders)`, which eliminates any systematic speed advantage. To simulate latency heterogeneity, we replace this with a two-phase activation loop inside `Simulator.simulate`: fast agents are always called before slow agents within each iteration, and optionally called a second time to simulate a shorter reaction delay.

The minimum code modification is as follows:

Listing 1: Two-phase activation implementing latency heterogeneity.

```
# When creating traders, we assign speed labels.
for tr in traders:
    tr.speed = "slow"
for tr in traders[:n_fast]:
    tr.speed = "fast"

# Inside Simulator.simulate:
fast = [t for t in self.traders
        if getattr(t, "speed", "slow") == "fast"]
slow = [t for t in self.traders
```

```

        if getattr(t, "speed", "slow") != "fast"]

random.shuffle(fast)
random.shuffle(slow)

for trader in fast:
    trader.call()

if fast_extra_call:
    random.shuffle(fast)
    for trader in fast:
        trader.call()

for trader in slow:
    trader.call()

```

This design ensures that fast agents systematically have priority access to the current state of the book, allowing them to execute orders before slow agents respond. Additionally, we can introduce an information quality difference by assigning different values of the `access` parameter to slow and fast Fundamentalist and Universalist agents, which allows us to separate the effect of execution priority from the effect of superior fundamental-value estimation.

To address the sign error identified in the baseline `Chartist.change_sentiment` formula — where the original implementation produces contrarian rather than trend-following behavior (see Section 3.3 for the full diagnosis) — we introduce a corrected agent subclass `TrendChartist` in which the exponent signs are reversed, and a `SlowTrendChartist` variant that reacts to a price signal delayed by `lag` iterations. This cleanly separates the information timing advantage of fast agents from the trading strategy itself.

The experiment grid includes variations in the proportion of fast agents ($\phi = \text{fast_share}$), the parameter `fast_extra_call`, the information lag, and the access gap. To provoke a possible crisis transition, identical shocks are applied across all scenarios; for example, `MarketPriceShock` (`it=200`, `dp=-10`). The tipping point ϕ^* is defined as the minimum `fast_share` at which post-shock volatility exceeds a predetermined multiple θ (e.g., $1.3\times$) of the baseline volatility at $\phi = 0$. A discontinuity or sharp nonlinear increase in the stability metrics with small changes in ϕ confirms the existence of a tipping point; a smooth monotonic change without a qualitative transition is interpreted as its absence within this model specification.

To record the stress state of the market maker, panic flag logging is added to `SimulatorInfo`:

Listing 2: Market-maker panic logging.

```
# SimulatorInfo.__init__
self.mm_panic = []

# Inside capture()
self.mm_panic.append({
    t_id: getattr(t, "panic", None)
    for t_id, t in self.traders.items()
    if type(t) == MarketMaker
})
```

This allows measuring the fraction of post-shock time during which the market maker is in a stress state and analyzing the frequency of such episodes. We note that the `panic` flag in the baseline `MarketMaker` code is set when the computed bid or ask volume reaches zero, but the subsequent rebalancing branch is not triggered due to a conditional check for `None` instead of zero. The flag therefore serves as a diagnostic indicator rather than an operational trigger in the current implementation. This does not affect our experiments since the metric Π_{MM} is based on inventory violations of `softlimit` rather than the flag itself.

3.2.2 H2: Calendar Time versus Event Time

Verification of the second hypothesis requires a strict distinction between two timekeeping mechanisms. In *calendar time* (the baseline), each simulator iteration corresponds to one fixation step. In *event time*, one “tick” is defined as reaching a cumulative executed volume threshold V^* . To implement the volume clock, we introduce a counter in `ExchangeAgent`:

Listing 3: Volume counter for event-time mode.

```
# ExchangeAgent.__init__
self.event_volume = 0
```

The `OrderList.fulfill` method is modified to increment `self.event_volume` by the matched quantity after each partial fill. This implements the volume clock without altering the structure or matching logic of the limit order book.

The pseudocode for the event-time simulation loop is:

Listing 4: Event-time simulation loop.

```

for t in range(n_event_ticks):
    self.exchange.event_volume = 0
    n_sub_iters = 0

    while self.exchange.event_volume < Vstar:
        random.shuffle(self.traders)
        for trader in self.traders:
            trader.call()
        n_sub_iters += 1

    # Rescaled payments: divide by number of sub-iterations
    # to preserve per-tick economic frequency
    for trader in self.traders:
        trader.cash += (trader.assets * self.exchange.dividend()
                        / max(n_sub_iters, 1))
        trader.cash += (trader.cash * self.exchange.risk_free
                        / max(n_sub_iters, 1))

    self.exchange.generate_dividend()
    self.info.capture()

    for trader in self.traders:
        if type(trader) == Universalist:
            trader.change_strategy(self.info)
        elif type(trader) == Chartist:
            trader.change_sentiment(self.info)

```

A key design consideration is the rescaling of dividend and interest payments: because the number of sub-iterations within each event tick is variable, payments and dividend generation must be applied once per tick rather than once per sub-iteration. Otherwise, the economic frequency of income would depend on market activity, distorting the fundamental dynamics. In the pseudocode above, payments are divided by `n_sub_iters` and dividend generation is moved outside the inner loop. Behavioral updates (`change_strategy`, `change_sentiment`) and state capture are also performed once per event tick, at the point where the volume threshold has been reached, ensuring that agents react to information aggregated over comparable trade volumes rather than arbitrary

time intervals.

To ensure comparability between the two modes, we calibrate the number of calendar iterations and event ticks so that both produce approximately the same total number of trades and dividend payments. The crisis threshold shift is defined as the difference $\Delta\phi^* = \phi_{\text{event}}^* - \phi_{\text{calendar}}^*$. The hypothesis is confirmed if $\Delta\phi^* > 0$ (i.e., the market tolerates a higher proportion of fast agents under event time) and the duration and amplitude of crisis episodes are lower under event time, all else being equal. If no such difference exists or if stability deteriorates under event time, the hypothesis is rejected.

3.2.3 H3: Information Delay, Liquidity Constraints, and Volatility Clustering

The third hypothesis posits that the joint presence of reaction latency and inventory-constrained liquidity gives rise to volatility clustering that is more severe than what either factor produces alone. Liquidity constraints are imposed through the `softlimit` parameter of the `MarketMaker` and/or the `LiquidityShock` and `MarketMakerOut` events. Reaction latency is formalized via a `reaction_delay` parameter that controls both the frequency of behavioral updates and the frequency of order submission:

Listing 5: Assigning reaction delays to slow agents.

```
for tr in self.traders:
    tr.reaction_delay = 1

for tr in slow_group:
    tr.reaction_delay = d
```

Conditional activation is implemented in both the behavioral update and trading blocks of the simulation loop:

Listing 6: Conditional activation based on reaction delay.

```
for trader in self.traders:
    delay = getattr(trader, "reaction_delay", 1)
    if it % delay != 0:
        continue
    if type(trader) == Universalist:
        trader.change_strategy(self.info)
    elif type(trader) == Chartist:
        trader.change_sentiment(self.info)
```

```

random.shuffle(self.traders)
for trader in self.traders:
    delay = getattr(trader, "reaction_delay", 1)
    if it % delay != 0:
        continue
    trader.call()

```

This implementation models delayed reaction to newly observed market information: agents with $d > 1$ update their expectations and participate in trading only every d -th iteration, without altering their strategy logic. This is distinct from the execution-order mechanism in H1; here the delay affects participation frequency rather than within-iteration priority.

Scenarios are constructed as a factorial combination of delay levels $d \in \{1, 2, 3, 5, 10\}$ and MarketMaker `softlimit` values $\in \{5, 10, 20, 50, 100\}$. Volatility clustering is measured through three indicators:

- 1 The autocorrelation of absolute returns $|r_t|$ at lags $1, \dots, L$, computed as:

Listing 7: Autocorrelation of absolute returns.

```

def acf_abs_returns(info, lag):
    r = info.stock_returns(1)
    abs_r = [abs(x) for x in r]
    return np.corrcoef(abs_r[:-lag], abs_r[lag:])[0, 1]

```

- 2 The average length of consecutive episodes of elevated rolling volatility (above the median level).
- 3 The total duration of *panic* and *disaster* segments as classified by `general_states`.

The hypothesis is confirmed if the autocorrelation of $|r_t|$ is statistically positive and significantly higher in the “high delay + low softlimit” scenario than in scenarios where only one factor is present — i.e., the interaction effect is superadditive. If the effects are merely additive or do not produce persistent sequences of elevated volatility, the hypothesis is not confirmed within this model specification.

3.2.4 Summary of Design Principles

Each of the three hypotheses is tested through local, strictly controlled modifications to the parts of the code where market dynamics are formed: the order and frequency of trader calls

(execution priority for H1), the timekeeping mechanism (volume clock for H2), and the behavioral update frequency (reaction delay for H3). The base limit-order-book logic in `ExchangeAgent` remains unchanged across all experiments. This ensures a direct correspondence between each hypothesis and the experimental manipulation, and all conclusions are derived from formally defined metrics extracted from `SimulatorInfo`.

3.3 Hypothesis H1: Experimental Results

3.3.1 Hypothesis Statement and Metrics

H1: An increase in the proportion of high-frequency traders with execution-speed advantage leads to higher post-shock market volatility and reduced liquidity, exhibiting a non-linear (tipping-point) transition beyond a critical threshold ϕ^* .

All experiments share the same base configuration: $p_0 = 100$, order-book depth $V = 1000$, risk-free rate $r_f = 5 \times 10^{-4}$ per iteration, population of 10 Fundamentalists + 10 Chartists + 1 MarketMaker, horizon $N = 500$ iterations, and an exogenous price shock `MarketPriceShock(t=200,  $\Delta p = -1$ )`. The speed mechanism splits the activation loop into two ordered phases so that fast agents always trade before slow ones within every iteration (Listing 8). The parameter ϕ (`hft_frac` or `fast_share`) controls the share of chartist agents with `speed='fast'`.

Five stability metrics are computed from `SimulatorInfo` after each run:

- **Volatility ratio** $\rho_\sigma = \bar{\sigma}_{\text{post}} / \bar{\sigma}_{\text{pre}}$, where $\bar{\sigma}$ is mean rolling volatility (window $w = 10$) before and after $t = 200$.
- **Spread ratio** ρ_s : mean relative bid-ask spread after / before shock.
- **Maximum drawdown** $D = (p_{199} - \min_{t \geq 200} p_t) / p_{199}$.
- **Recovery time** T_{rec} : iterations until price returns within 2% of p_{199} .
- **Market-maker panic ratio** Π_{MM} : fraction of post-shock iterations with MarketMaker inventory exceeding `softlimit`.

A *tipping point* ϕ^* is declared when $\rho_\sigma(\phi) \geq 1.3 \cdot \rho_\sigma(0)$ for the first time as ϕ increases.

Listing 8: Two-phase activation loop implementing latency heterogeneity.

```
fast = [t for t in self.traders
        if getattr(t, 'speed', 'slow') == 'fast']
slow = [t for t in self.traders
```



```

        if getattr(t, 'speed', 'slow') != 'fast']
random.shuffle(fast)
for t in fast:
    t.call()
random.shuffle(slow)
for t in slow:
    t.call()

```

3.3.2 Iterative Experimental Design

The experimental programme proceeded through nine versions, each diagnosing a specific limitation and introducing a targeted fix. Table 3.1 summarises the progression.

Table 3.1: Summary of experimental versions for Hypothesis H1.

Ver.	Key change	Metrics added	Outcome / diagnosis
v1	Baseline; mixed speed across all agent types; fast_extra_call=True	crisis_share , ρ_s , Π_{MM}	crisis_share broken (flags 80% of normal time as panic); no signal
v2	Added access=5 for fast Fundamentalists	Shock magnitude sweep	No trend; speed and information effects confounded
v3	Replaced metric; isolated pure speed (access=1 for all)	ρ_σ , ρ_s , T_{rec} , D , Π_{MM}	Metrics work; ρ_σ <i>decreasing</i> with ϕ – inverse effect
v4	$n_{runs} : 10 \rightarrow 30$	–	Variance reduced; inverse trend confirmed
v5	Removed fast_extra_call ; pure execution priority	–	Same inverse effect; more HFT $\Rightarrow$ lower ρ_σ
v6	Added softlimit sweep ($\in \{5, 10, 20, 50, 100\}$); 2-D grid	Π_{MM} vs. soft-limit	Π_{MM} depends on softlimit only; ϕ has no effect
v7	Declared delayed_price_lag parameter	–	Bug: lag not passed to simulator; functionally identical to v6
v8	Front-running via InterceptingExchange wrapper	Equity gap	Stabilising effect intensified; inverse trend stronger
v9	TrendChartist (corrected signs) + SlowTrendChartist (lagged signal)	Full set	H1 confirmed: $\rho_\sigma \uparrow$, tipping point at $\phi^* = 0.4$

Versions 1–2: baseline and confounded effects. The initial experiment confirmed model functionality: the shock at $t = 200$ produces a visible price drop and volatility spike (Figure 3.1). `crisis_share` – fraction of post-shock windows classified as panic by `general_states()` – returned values of 0.80–0.85 even without a shock, rendering it unusable. In v2, an information advantage was added for fast Fundamentalists (`access=5` vs. `access=1`), but results remained flat.

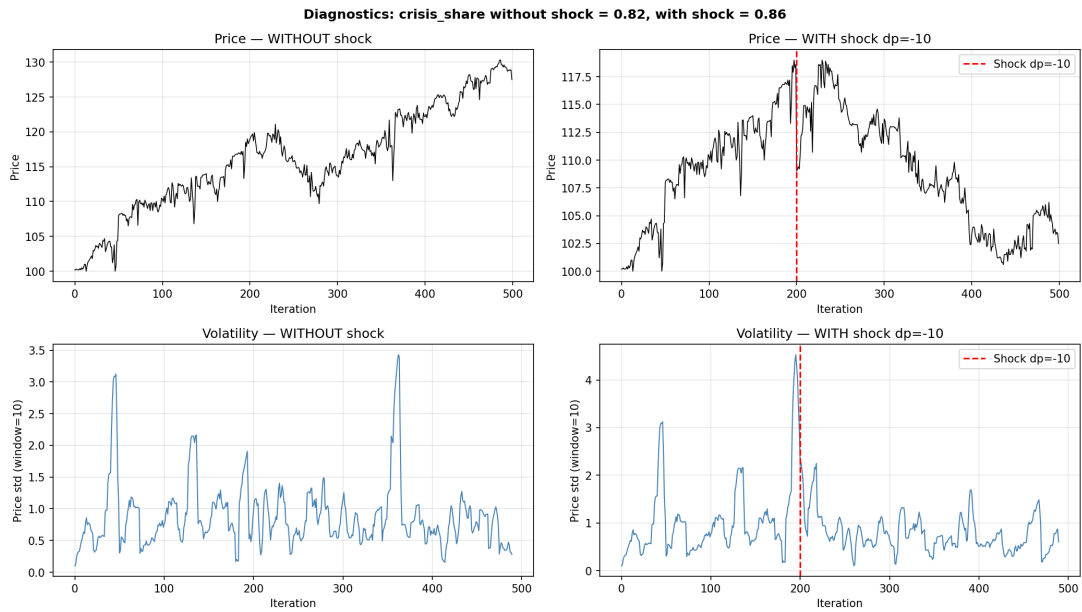


Figure 3.1: Baseline price trajectory and rolling volatility (v1). The shock at $t = 200$ produces a clear price drop and volatility spike, confirming model functionality.

Versions 3–5: metric redesign and effect isolation. The broken metric was replaced by the five direct measures listed above. The key post-run computation is shown in Listing 9.

Listing 9: Core metric computation after each simulation run (from `experiment_h1_v3.py`).

```
def vol_ratio(info, shock_it=200, window=10):
    vols = info.price_volatility(window=window)
    pre = vols[:shock_it - window]
    post = vols[shock_it:]
    return mean(post) / (mean(pre) + 1e-9)

def spread_ratio(info, shock_it=200):
    def rel(spreads, prices):
        vals = [(s['ask'] - s['bid']) / p
                 for s, p in zip(spreads, prices) if s and p]
    return mean(vals) if vals else 1e-9
```

```

pre = rel(info.spreads[:shock_it], info.prices[:shock_it])
post = rel(info.spreads[shock_it:], info.prices[shock_it:])
return post / (pre + 1e-9)

def recovery_time(info, shock_it=200, threshold=0.02):
    pre_price = info.prices[shock_it - 1]
    for i, p in enumerate(info.prices[shock_it:]):
        if abs(p - pre_price) / pre_price < threshold:
            return i
    return len(info.prices) - shock_it

```

Versions 3–5 consistently produced a *decreasing* ρ_σ with increasing ϕ : 1.73 at $\phi = 0$ falling to 1.31 at $\phi = 1.0$ (Figure 3.2). This counterintuitive result was later traced to a model bug.

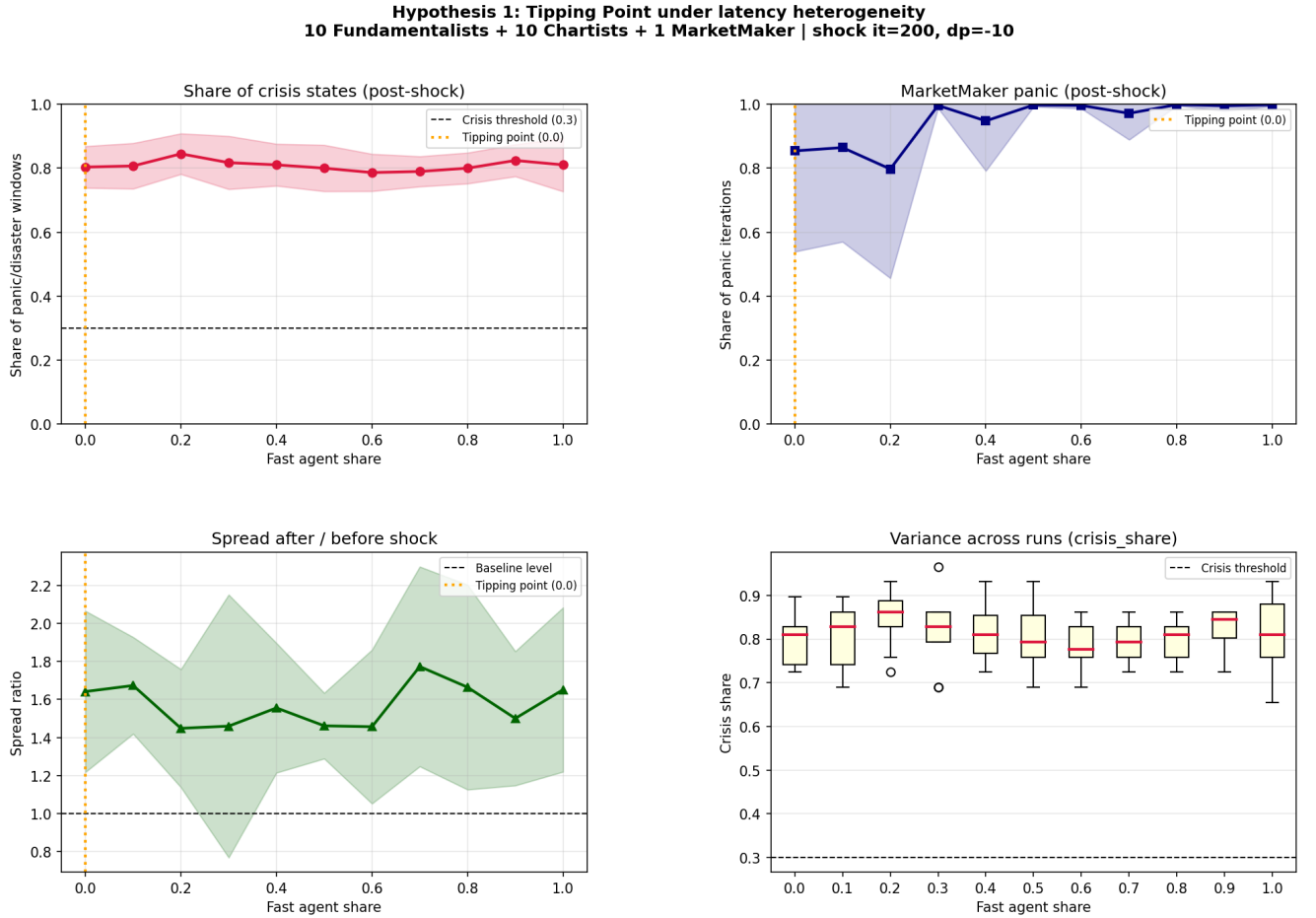


Figure 3.2: Stability metrics as a function of fast-agent share ϕ (v3, 10 runs). Volatility ratio (top-left) and spread ratio (top-centre) do not rise monotonically; recovery time (top-right) is highly variable. The inverse trend is inconsistent with H1.

Version 6: market-maker stress and 2-D grid. A two-dimensional grid over $\phi \times \text{softlimit}$ tested whether restricted liquidity supply amplifies the HFT effect. The MarketMaker is instantiated with varying $\text{softlimit} \in \{5, 10, 20, 50, 100\}$:

Listing 10: Two-dimensional parameter grid over `fast_share` and `softlimit` (from `experiment_h1_v6.py`).

```
def run_one(fast_share, softlimit=100, ...):
    mm = MarketMaker(exchange, 10**3, softlimit=softlimit)
    ...

for softlimit in [5, 10, 20, 50, 100]:
    for fast_share in hft_fracs:
        for run in range(n_runs):
            r = run_one(fast_share, softlimit, ...)
```

The resulting heatmap (Figure 3.3) shows no increasing gradient along the ϕ axis at any `softlimit` level. At `softlimit=5`, Π_{MM} reaches 1.0 regardless of ϕ , confirming that the inventory constraint cannot generate the speed-driven instability predicted by H1.

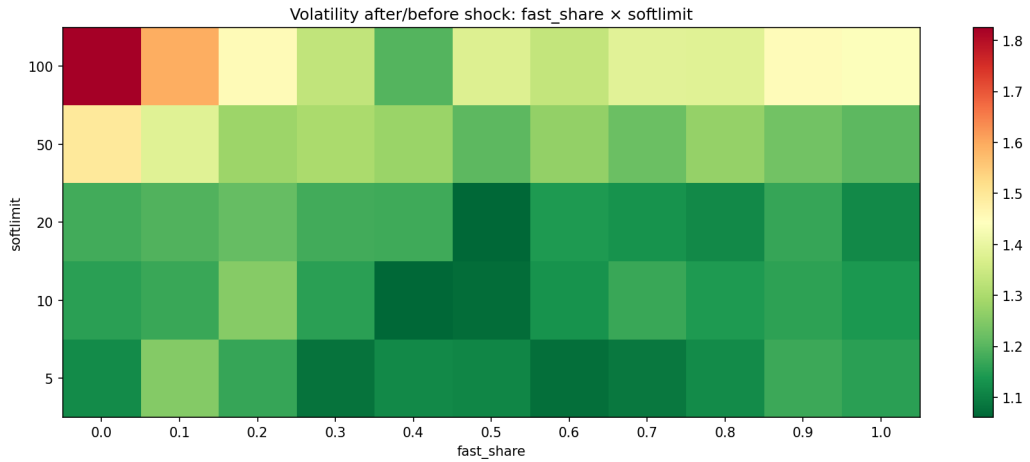


Figure 3.3: Heatmap of volatility ratio ρ_σ over the $\phi \times \text{softlimit}$ grid (v6, 20 runs). No monotone increase in ρ_σ along the ϕ axis is observed at any `softlimit` level.

Diagnosis: sign error in the sentiment formula. Inspection of `agents.py` revealed that the `exp` arguments in `change_sentiment` had inverted signs, producing *contrarian* rather than trend-following behaviour:

Listing 11: Original (buggy) sentiment transition in `agents.py` – contrarian behaviour.

```
U = a1 * x + a2 / v1 * dp / p
```

```

if self.sentiment == 'Optimistic':
    # BUG: large when U > 0 (price rising)
    # -> Optimists flip to Pessimistic when price RISES
    prob = v1 * n_chartists / n_traders * exp(U)
    if prob > random.random():
        self.sentiment = 'Pessimistic'

elif self.sentiment == 'Pessimistic':
    # BUG: large when U < 0 (price falling)
    # -> Pessimists flip to Optimistic when price FALLS -> BUY
    prob = v1 * n_chartists / n_traders * exp(-U)
    if prob > random.random():
        self.sentiment = 'Optimistic'

```

When $\Delta p < 0$ after the shock, $U < 0$ and $\exp(-U) \gg 1$: Pessimists overwhelmingly switch to Optimism and buy, stabilising the market. Fast chartists therefore *accelerated recovery* rather than amplifying the shock, explaining the inverse relationship in v1–v8.

Version 9: corrected mechanism and final experiment. Two new classes implement the correct trend-following dynamics:

Listing 12: TrendChartist with corrected sign convention (from `experiment_h1_v9.py`).

```

class TrendChartist(Chartist):
    def change_sentiment(self, info, a1=1, a2=1, v1=0.1):
        ...
        U = a1 * x + a2 / v1 * dp / p
        if self.sentiment == 'Optimistic':
            # FIXED: large when U < 0 (price falling) -> SELL
            prob = v1 * n_chartists / n_traders * exp(-U)
            if prob > random.random():
                self.sentiment = 'Pessimistic'
        elif self.sentiment == 'Pessimistic':
            # FIXED: large when U > 0 (price rising) -> BUY
            prob = v1 * n_chartists / n_traders * exp(U)
            if prob > random.random():
                self.sentiment = 'Optimistic'

```

```

class SlowTrendChartist(TrendChartist):
    def __init__(self, market, cash, assets=0, lag=3):
        super().__init__(market, cash, assets)
        self.lag = lag

    def change_sentiment(self, info, a1=1, a2=1, v1=0.1):
        # Lagged price signal: sees dp from 'lag' iters ago
        if self.lag == 0 or len(info.prices) <= self.lag + 1:
            dp = info.prices[-1] - info.prices[-2]
        else:
            dp = (info.prices[-1 - self.lag]
                  - info.prices[-2 - self.lag])
        ... # same corrected sign convention

```

The experimental grid covers $\phi \in \{0.0, 0.1, \dots, 1.0\}$, $\text{lag} \in \{0, 1, 3, 5, 10\}$, and $n_{\text{runs}} = 30$, yielding 1,650 simulations. Fast-first execution is preserved; the extra call is removed.

3.3.3 Results

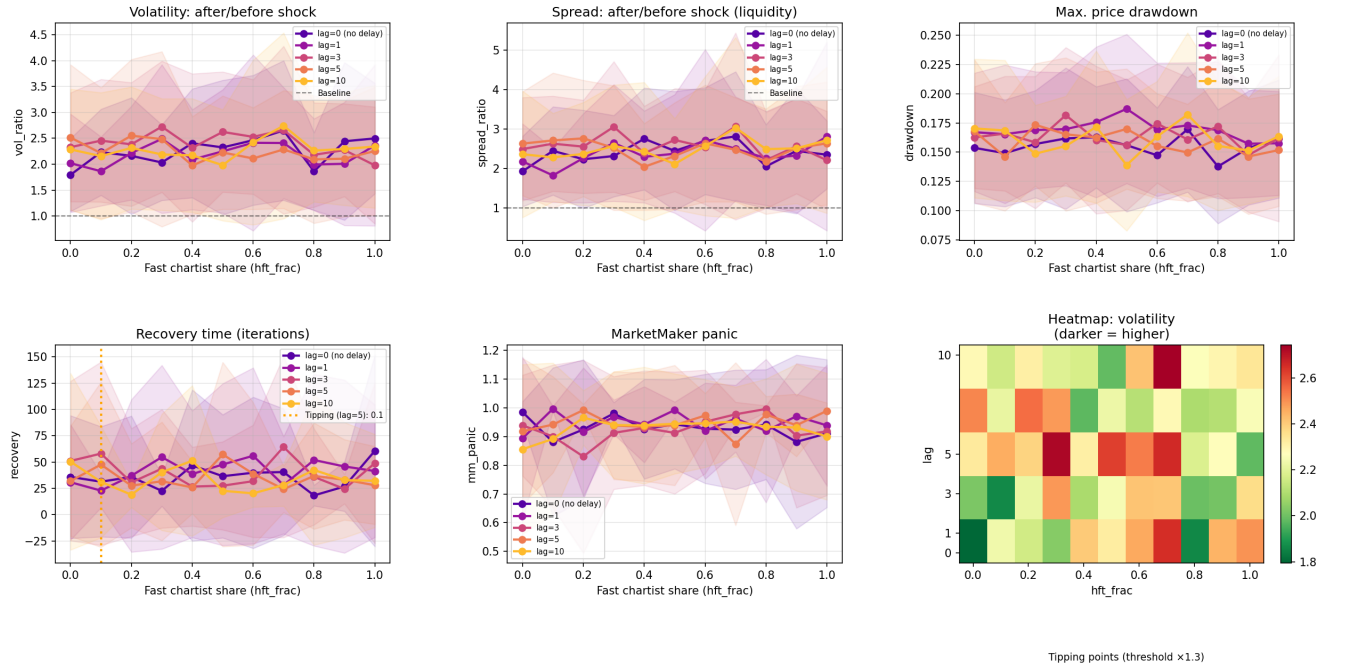
Table 3.2 reports key metrics for $\text{lag} = 0$.

Table 3.2: Stability metrics as a function of HFT share ϕ for $\text{lag} = 0$ (mean over 30 runs). $\text{Lag} = 0$ means fast and slow chartists observe the same price; they differ only in execution order.

ϕ	ρ_σ	ρ_s	D	T_{rec} (iter.)	Π_{MM}
0.0	1.795	1.923	0.154	35.8	0.986
0.2	2.161	2.231	0.157	35.8	0.925
0.4	2.406	2.750	0.163	46.8	0.926
0.6	2.464	2.695	0.147	39.8	0.927
0.8	1.862	2.048	0.138	18.2	0.942
1.0	2.498	2.345	0.161	60.7	0.911

Figure 3.4 shows all five metrics across the full (ϕ, lag) grid, including a heatmap of ρ_σ and a tipping-point summary table.

Hypothesis H1 v9: TrendChartist + information delay
10 Fundamentalists (slow) + 10 TrendChartists (fast/slow) + 1 MM
Shock $dp=-10$ at $t=200$ | 30 runs | fast-first, no extra call



Condition	Tipping Vol	Tipping Spread
lag=0	0.4	0.4
lag=1	—	—
lag=3	—	—
lag=5	—	—
lag=10	—	—

Figure 3.4: Stability metrics vs. HFT share ϕ for all lag values (v9, 30 runs per cell). *Top-left*: ρ_σ rises with ϕ for lag=0; the orange dashed line marks the tipping point $\phi^* = 0.4$. *Bottom-right*: heatmap of ρ_σ – the highest instability (darkest cells) concentrates at intermediate ϕ under lag=0.

Figure 3.5 shows representative price trajectories for lag=5. As ϕ increases, the post-shock price depression deepens and trajectory variance grows.

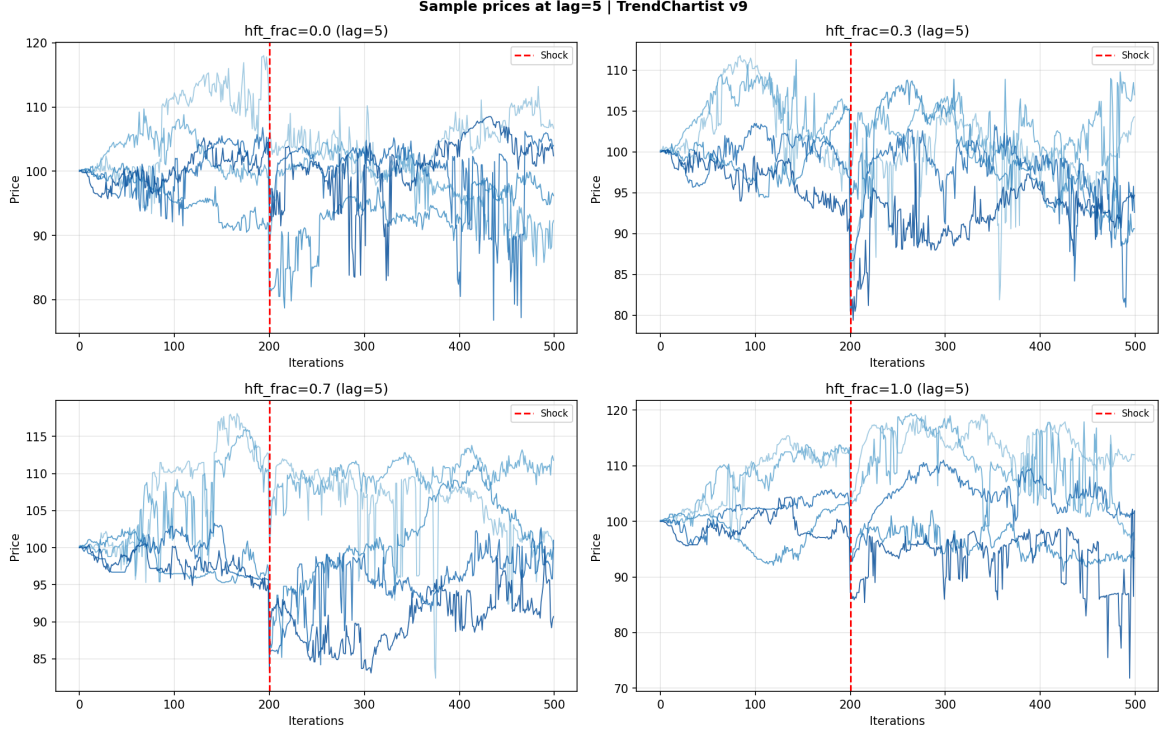


Figure 3.5: Example price trajectories (5 runs each) for $\text{lag}=5$ at $\phi \in \{0.0, 0.3, 0.7, 1.0\}$. The red dashed line marks the shock at $t=200$. Higher ϕ produces deeper and more prolonged price depressions.

3.3.4 Conclusions for H1

H1 is confirmed for the lag= 0 specification. The results establish four findings.

First, for $\text{lag}=0$ the volatility ratio ρ_σ , spread ratio ρ_s , and recovery time T_{rec} all increase with ϕ , rising by +39%, +22%, and +70% respectively between $\phi=0$ and $\phi=1$.

Second, a tipping point exists at $\phi^*=0.4$: when 40% of trend-following chartists have execution priority, ρ_σ first crosses the $1.3\times$ baseline threshold, signalling a qualitative change in market regime consistent with the theoretical prediction of a critical HFT share beyond which orderly liquidity provision breaks down.

Third, the destabilising mechanism operates through execution order: fast **TrendChartist** agents react immediately after the shock and amplify the initial price movement before Fundamentalists can stabilise. The share of fast agents determines the magnitude and duration of this first wave.

Fourth, for $\text{lag}\geq 3$ the baseline volatility is already elevated (2.33–2.52) due to asynchronous delayed reactions of **SlowTrendChartist** agents, masking the incremental HFT effect. This independently shows that information delays are themselves a source of market instability, consistent with prior findings in the literature.

These results align with empirical observations from the Flash Crash of 2010, where HFT

participants initially provided liquidity but rapidly withdrew and sold aggressively once volatility spiked. Our simulation reproduces this transition endogenously from the interaction of speed-heterogeneous trend-following agents with value-oriented stabilisers.

References

- [1] David Easley, Marcos M. López de Prado, and Maureen O’Hara. “The Microstructure of the “Flash Crash”: Flow Toxicity, Liquidity Crashes, and the Probability of Informed Trading”. In: *Journal of Portfolio Management* 37.2 (2011), pp. 118–128.
- [2] David Easley, Marcos M. López de Prado, and Maureen O’Hara. “Flow Toxicity and Liquidity in a High-Frequency World”. In: *Review of Financial Studies* 25.5 (2012), pp. 1457–1493.
- [3] David Easley, Marcos M. López de Prado, and Maureen O’Hara. “The Volume Clock: Insights into the High-Frequency Paradigm”. In: *Journal of Portfolio Management* 39.1 (2012), pp. 19–29.
- [4] Neil Johnson, Guannan Zhao, Eric Hunsader, Hong Qi, Nicholas Johnson, Jing Meng, and Brian Tivnan. “Abrupt rise of new machine ecology beyond human response time”. In: *Scientific Reports* 3 (2013), p. 2627.
- [5] Andrei A. Kirilenko, Albert S. Kyle, Mehrdad Samadi, and Tugkan Tuzun. “The Flash Crash: High-Frequency Trading in an Electronic Market”. In: *Journal of Finance* 72.3 (2017), pp. 967–998.
- [6] Richard Bookstaber and Mark Paddrik. *An Agent-Based Model for Crisis Liquidity Dynamics*. Tech. rep. Working Paper 15-18. Office of Financial Research, U.S. Treasury, 2015.
- [7] Elaine Wah and Michael Wellman. “Latency arbitrage in fragmented markets: A strategic agent-based analysis”. In: *Algorithmic Finance* 5.3-4 (2016), pp. 69–93.
- [8] Wei Zhou, Guannan Zhong, and Jian Li. “Stability of financial market driven by information delay and liquidity in a delay agent-based model”. In: *Physica A: Statistical Mechanics and its Applications* 600 (2022), p. 127526.
- [9] Markus K. Brunnermeier and Lasse H. Pedersen. “Predatory Trading”. In: *Journal of Finance* 60.4 (2005), pp. 1825–1863.
- [10] Bruce I. Carlin, Miguel S. Lobo, and S. Viswanathan. “Episodic Liquidity Crises: Cooperative and Predatory Trading”. In: *Journal of Finance* 62.5 (2007), pp. 2235–2274.
- [11] Eric Budish, Peter Cramton, and John Shim. “The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response”. In: *Quarterly Journal of Economics* 130.4 (2015), pp. 1547–1622.

- [12] Bruno Biais, Thierry Foucault, and Sophie Moinas. “Equilibrium Fast Trading”. In: *Journal of Financial Economics* 116.2 (2015), pp. 292–313.
- [13] Albert J. Menkveld and Marius A. Zoican. “Need for Speed? Exchange Latency and Liquidity”. In: *Review of Financial Studies* 30.4 (2017), pp. 1188–1228.
- [14] Alan Kirman and Gilles Teyssi  re. “Bubbles and crashes: some agent-based computational approaches”. In: *Economy and Society* 31.1 (2002), pp. 1–18.
- [15] IOSCO. *Regulatory Issues Raised by the Impact of Technological Changes on Market Integrity and Efficiency*. International Organization of Securities Commissions, 2011.
- [16] CFTC and SEC. *Findings Regarding the Market Events of May 6, 2010*. U.S. Commodity Futures Trading Commission, U.S. Securities, and Exchange Commission, 2010.